

Greedy Algorithms (Optimization problems)

- These algos are used for optimization problems
- especially for to find minima or maxima

Bangalore (A) → Delhi (B)

① Solution Space

- on Road / Drive
- Train
- Bus
- flight
- cycling
- Bike

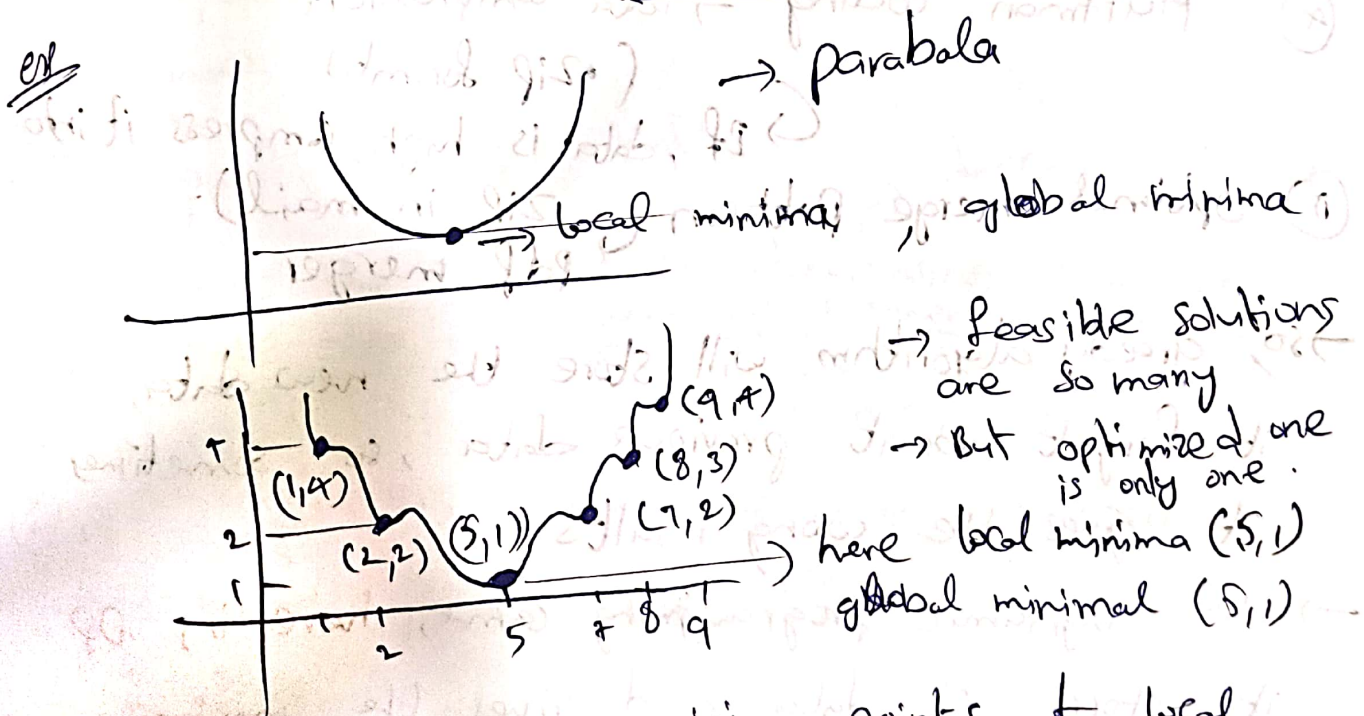
→ here, for minimum cost is constraint
so, Bus, flight cost is high

→ Another constraint minimum time.

② feasible solutions are Train, Bike, cycling

→ so, cycling, Bike will take time.

→ finally, single optimized solution ③



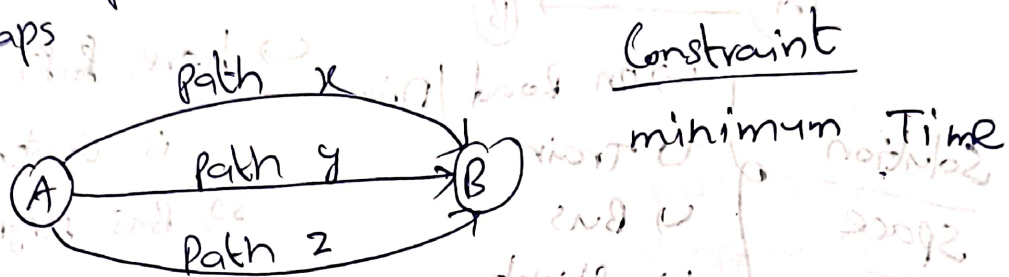
we have so many minimum points at local
but, global minima → which is lesser compared
to other points is (5,1) only.

→ In some cases, Greedy algo also failed.

So Deep Dynamic programming come into picture

Applications of Greedy algorithm

① shortest path (minimum time, minimum distance)
Maps



Maps Dijkstra's algorithm is used behind

the google maps

② Job scheduling

③ Fractional Knapsack

④ Huffman coding → Data compression

⑥ minimum spanning tree
↓
Kruskal's prim's algo.
⑦ single source shortest path
↓
Dijkstra's algo

(.zip format)

⑧ optimal merge pattern (zip in mail).
↓
pdf merger

→ So, greedy algorithm will store the new data, it forget about previous data, so, sometimes it gives the wrong results.

→ Then Dynamic programming came, here in DP it stores all data and gives the minima or maxima. means, best optimized version it gives. So, it takes some extra space.

→ space Complexity ↑ but Time Complexity is ↓
so, it is good to go.

→ It always provides best optimized version
but greedy sometimes fails.

Fractional Knapsack :-

objects	n_1	n_2	n_3	n_4	n_5	n_6	n_7
profit	25	75	100	50	45	90	80
weight	5	10	12	4	7	9	3

Constraint - maximum weight you can carry is 37
so, Total weight of object $\leq M.W$

get, Maximum profit → optimized solution

→ so, which objects you will take, which get more profit

Steps :-

① profit/weight ratio

→ decreasing order

② sort the objects in decreasing order

③ take objects in sequential manner $\sum w_i \leq 37$

so, P/w $n_1, n_2, n_3, n_4, n_5, n_6, n_7$

② $\Rightarrow$ $\frac{25}{5}, 7.5, 8.3, 12.5, 6.4, 10, 10$

→ so, take greatest P/w

$n_4 \rightarrow 12.5, n_6 \rightarrow 10, n_7 \rightarrow 10, n_3 \rightarrow 8.3$

	n_4	n_6	n_7	n_3	n_2	n_5	n_1
profit	50	90	30	100	75	45	25
weight	4	9	3	12	10	7	5
P/w	12.5	10	10	8.3	7.5	6.4	5

③

Fractions

1 1 1 1 9/10 0 0

Object	Net weight	profit
	$M=37$	
N_4	$37-4=33$	50
N_6	$33-9=24$	$50+90=140$
N_7	$24-3=21$	$140+30=170$
N_3	$21-12=9$	$170+100=270$
N_2	$9-\frac{9}{10}=0.9$	$270+100=370$ $270+\frac{9}{10} \times 75$ $\hookrightarrow 337.5$

$$\sum_{i=1}^n x_i \times w_i = (37)M$$

maximum profit

time complexity

step ① $\rightarrow n$

step ② $\rightarrow n \log n$

step ③ $\rightarrow n$

$$\Rightarrow n + n \log n + n \Rightarrow O(n \log n)$$

Implementation :-

def fractionalKnapsack(profit, weight, M):

fractions = [0] * len(profit)

objects = list(range(len(profit)))

① $\leftarrow$ ratio = [profit/weight for profit, weight in zip(profit, weight)]

② $\leftarrow$ objects.sort(key=lambda i: ratio[i], reverse=True)

$\hookrightarrow O(n \log n)$



③ # to take the objects sequentially, decreasing the weight & increasing the profit

$n = \text{len}(\text{profit})$

$\text{maxprofit} = 0$

for i in objects:

$\Rightarrow O(n)$

if $\text{weight}[i] \leq M$:

$M -= \text{weight}[i]$

$\text{maxprofit} += \text{profit}[i]$

$\text{fractions}[i] = 1$

else:

$\text{maxprofit} += \text{profit}[i] \times \frac{M}{\text{weight}[i]}$

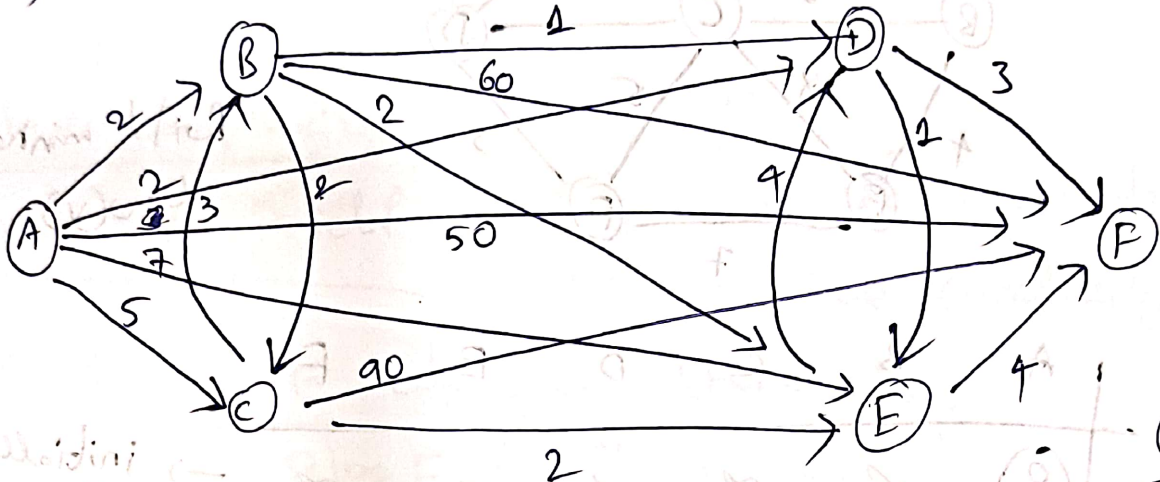
$\text{fractions}[i] = M / \text{weight}[i]$

break

return $(\text{maxprofit}, \text{fractions})$

Dijkstra's Algorithm

- application of greedy algorithm
- It is single source shortest path

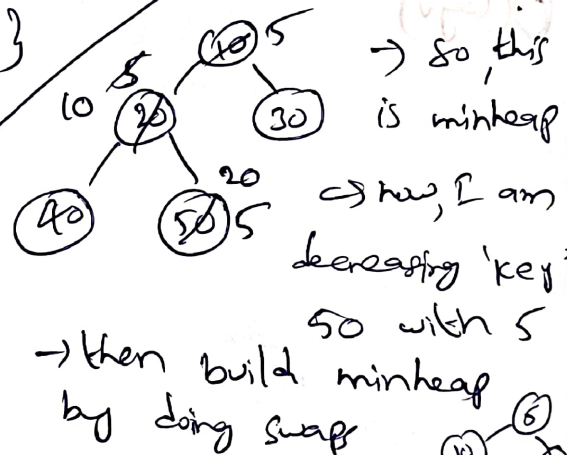


Source = A

the edge weights

- A: { B:2, C:5, D:2, E:7, F:50 }
- B: { C:2, D:2, E:2, F:60 }
- C: { B:3, E:2, F:90 }
- D: { E:2, F:3 }
- E: { D:4, F:4 }
- F: { }

Decrease key

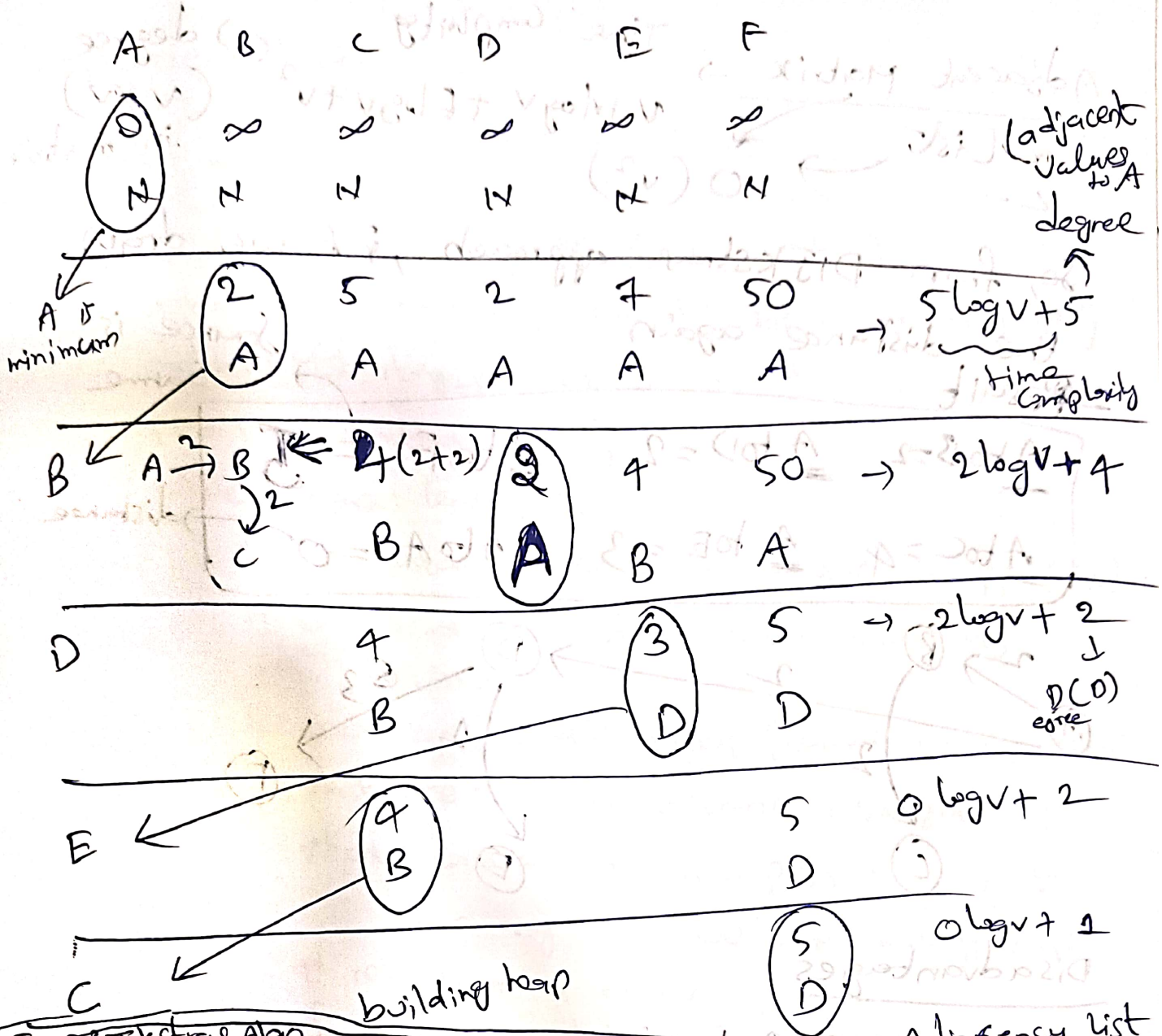


A	B	C	D	E	F	
0	∞	∞	∞	∞	∞	→ distance from
N	N	N	N	N	N	→ vertex

→ Create a minheap, delete

$O(n)$
 $O(V)$

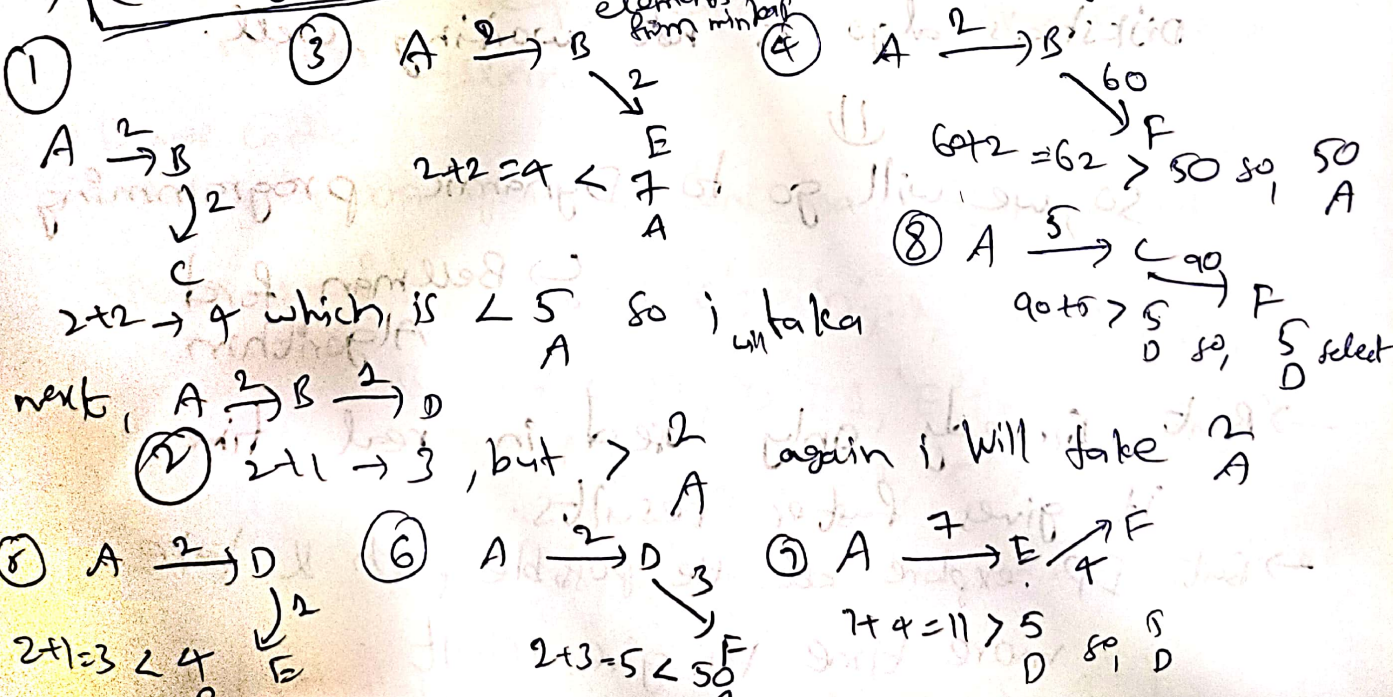
$O(n \log n)$
 $O(V \log V)$



DIJKSTRA'S ALGO

Time Complexity: $V + V \log V + E \log V + 2E$ (sum of degree)

$(V+E) \log V$

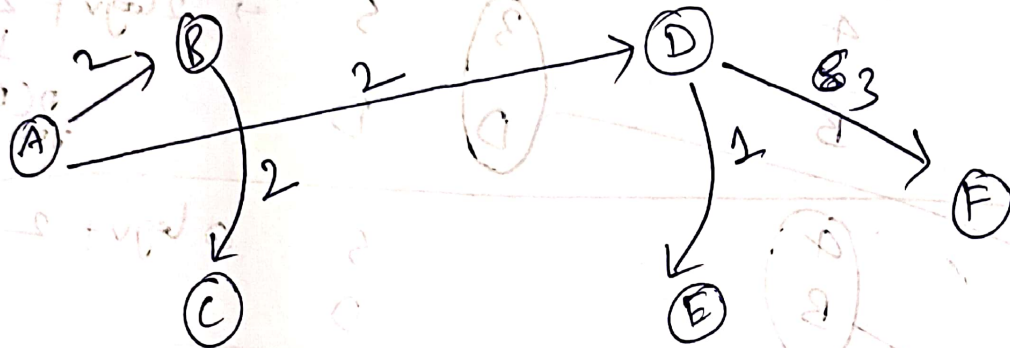


Adjacent Matrix $\rightarrow$ Time complexity $\rightarrow$ degree $(V \times V)$ in matrix
 or List $\rightarrow O(V^2)$

So, from Dijkstra's approach, if we draw the distance again

Result

A to B = 2 A to D = 2 A to F = 5
 A to C = 4 A to E = 3 A to A = 0



Disadvantages

$\hookrightarrow$ when Dijkstra's algo

$\rightarrow$ when we have negative edge case,

Dijkstra's algo is not working well.

$\Downarrow$
 so, we will go to Dynamic programming

$\hookrightarrow$ Bellman ford Algorithm

$\rightarrow$ But, Greedy only used in real time.

it gives faster results.

$\rightarrow$ But, DP explore all the possible path & take more time to give result.

Implementation

```
import heapq
```

```
def calculate_distances(graph, starting_vertex):
```

```
    distances = {vertex: float('inf') for vertex in graph}
```

```
    distances[starting_vertex] = 0
```

```
    pq = [(0, starting_vertex)]
```

```
    while pq: len(pq) > 0
```

```
        current_distance, current_vertex = heapq.heappop(pq)
```

```
        if current_distance > distances[current_vertex]:
```

```
            continue
```

```
        for neighbour, weight in graph[current_vertex].items():
```

```
            distance = current_distance + weight
```

```
            # only consider this new path if its better  
            # than any path we have already found
```

```
            # relaxation of edges
```

```
            if distance < distances[neighbour]:
```

```
                distances[neighbour] = distance
```

```
                heapq.heappush(pq, (distance, neighbour))
```

```
    return distances
```

```
# Driver Code
```

```
# Storing of the graph is done via adjacency list
```

```
distances = {'A': {'B': 2, 'C': 5, 'D': 1, 'E': 7, 'F': 50},
```

```
              'B': {'C': 2, 'D': 1, 'E': 2, 'F': 60},
```

```
              'C': {'B': 3, 'E': 2, 'F': 90},
```

```
              'D': {'E': 1, 'F': 3}, 'E': {'D': 4, 'F': 4},
```

```
              'F': {}}
```

```
print(calculate_distances(distances, 'A'))
```

Huffman coding :- (Data Compression)

zip →

a = 5

b = 9

c = 12

d = 13

e = 16

f = 45

character → frequency

each character to store 1 Byte
1 Byte = 8 bits

$$5 \times 8 + 9 \times 8 + 12 \times 8 + 13 \times 8 + 16 \times 8 + 45 \times 8 \\ = 40 + 72 + 96 + 104 + 128 + 360 \\ = 800 \text{ bits}$$

More frequency character → give lesser bit

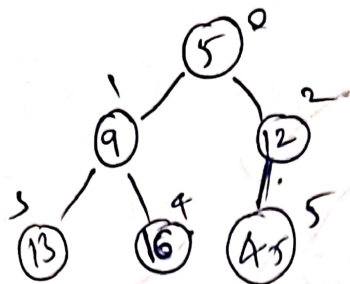
Less frequency character → give larger bit

1) Build heap → $O(n)$

↳ min heap

2) pop 2 times & insert the addition of two ^{deleted} items inside the min heap

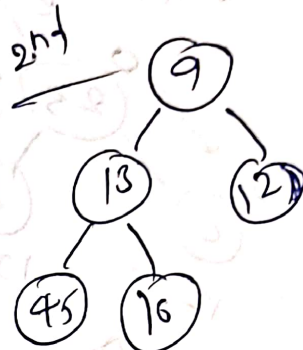
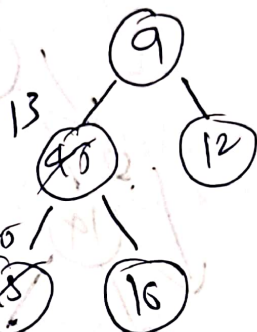
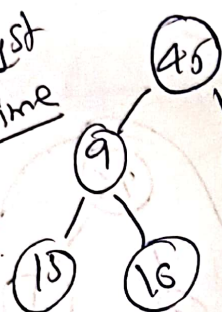
→ Build heap



→ Deletion 2 times

→ 5 will come.

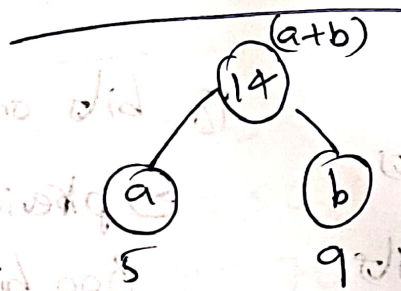
1st time



(13, 45, 16, 12)
will there

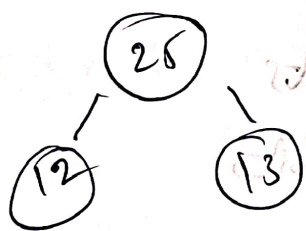
→ 2nd time deletion → 9 will come

Huffman Tree



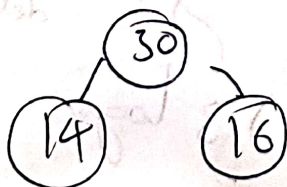
Now, 14 will add to
(13, 45, 16, 12, 14)
↓
(45, 16, 14)

Now, top 2 elements deleted — 13, 12



Now, 25 add to (45, 16, 14, 25)
↓
(45, 25)

Now, top 2 element deleted — 16, 14



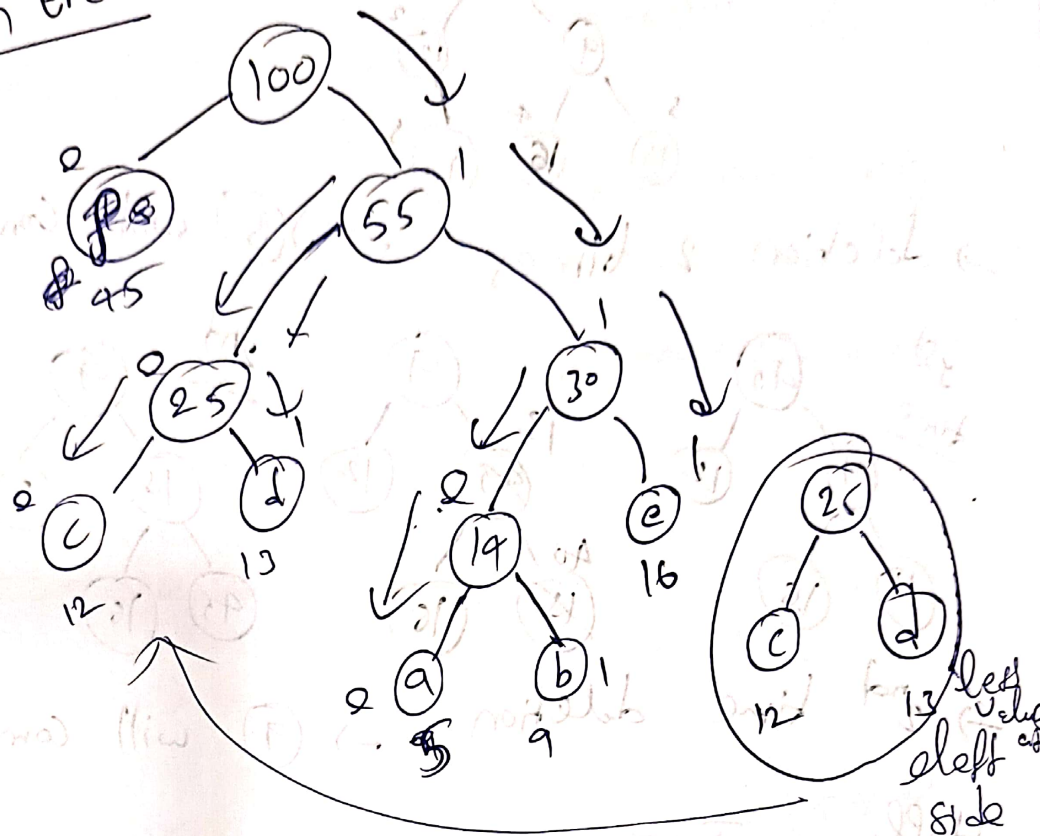
30 add to (45, 25, 30)
↓
(45, 55)

Now delete 2 elements



55 add to (45, 55) →
100 will add

Huffman tree



Coded values for

$f \rightarrow 0$ - 1 bit

$a \rightarrow 1100$ - 4 bits

$b \rightarrow 1101$ - 4 bits

$c \rightarrow 100$ - 3 bits

$d \rightarrow 101$ - 3 bits

$e \rightarrow 111$ - 3 bits

18 bits only
Previously 800 bits.

Time Complexity:

minheap build - $O(n)$

deletion $\rightarrow O(\log n)$

insertion $\rightarrow O(\log n)$

2 times deletion

$\rightarrow (n-1) 3 \log n$

2 times insertion

$$= O(n \log n)$$

→ In python

$$n \rightarrow j$$

8250

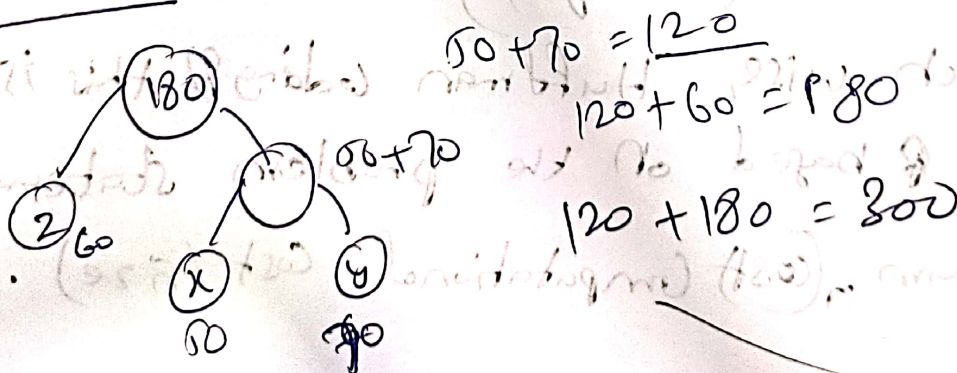
yz 70

$$Z = 60$$

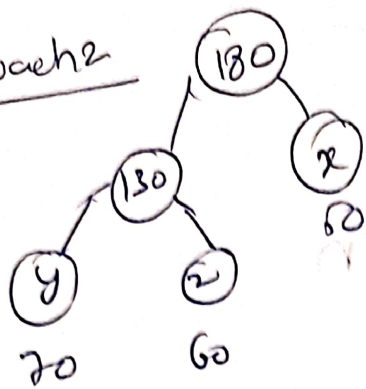
} → merging files

objective :- minimize the computational cost of merging all the given files.

Approach 1



Approach 2

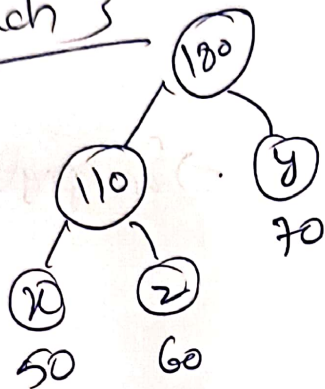


$$70 + 60 = 130$$

$$130 + 50 = 180$$

$$130 + 180 = 310$$

Approach 3



$$50 + 60 = 110$$

$$110 + 70 = 180$$

$$110 + 180 = 290$$

In approach 3, we are getting minimum Computational Cost.

1) Build heap $\rightarrow O(n)$

2) Pop 2 elements $\rightarrow (n-1) \times 3 \log n$

↳ addition of those 2 popped elements

& insertion in the min heap.

$$T.C = O(n) + (n-1) \times 3 \log n$$

$$= O(n \log n)$$

→ Approach wise, Huffman coding & this is same but, based on the problem statement minimum (Cost) Computational Cost (size).

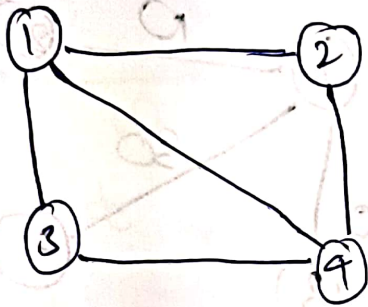
Spanning tree

Greedy

→ spanning tree is subgraph of graph G

- Properties
- 1) all vertices → available
 - 2) $(n-1)$ edges to connect the graph
(minimum) n - No. of vertices
 - 3) No cycle

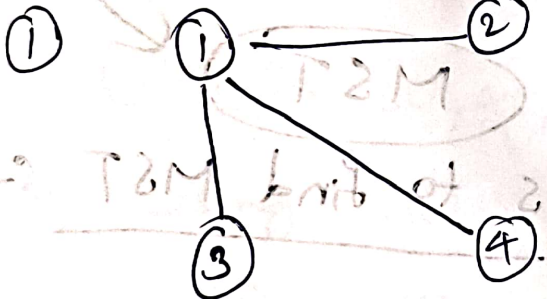
ex:-



$n = 4$

→ original graph

spanning trees:-

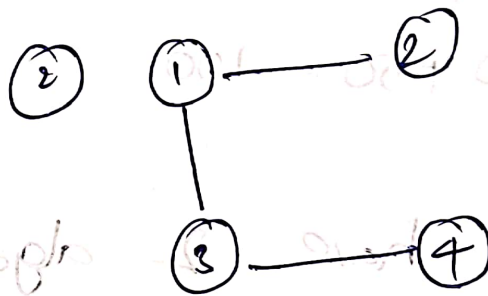


edges = 3

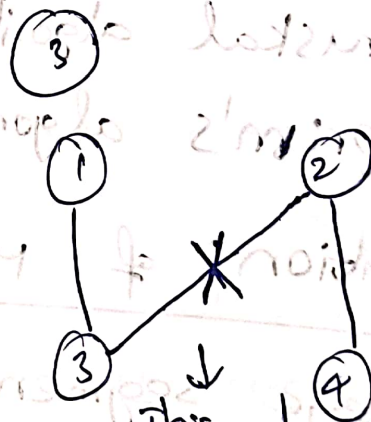
all vertices are available

No cycle

so, valid spanning tree



→ valid



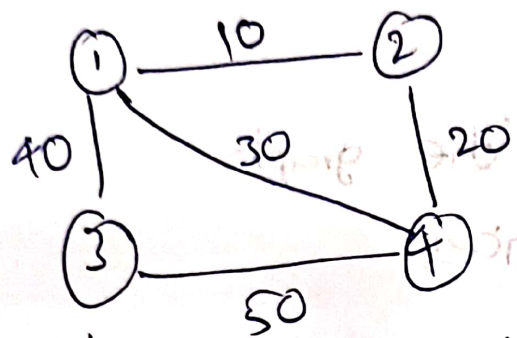
→ not valid

→ for a complete graph

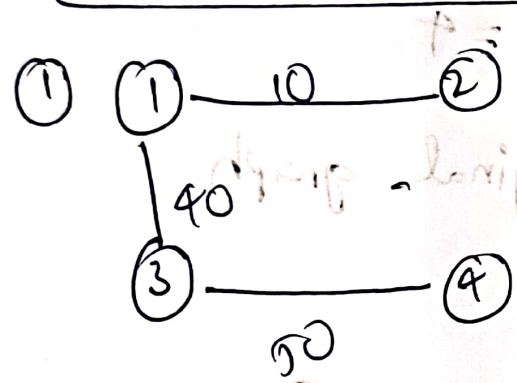
No. of spanning trees = $\frac{n-2}{n}$ In original graph

MST

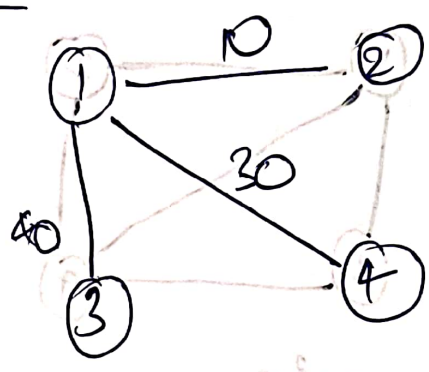
Minimum spanning tree :-



possible spanning trees



$$10 + 40 + 50 = 100$$



$$10 + 40 + 30 = 80$$

MST

So, we have 2 algorithms to find MST :-

- 1) Kruskal algorithm
- 2) Prim's algorithm

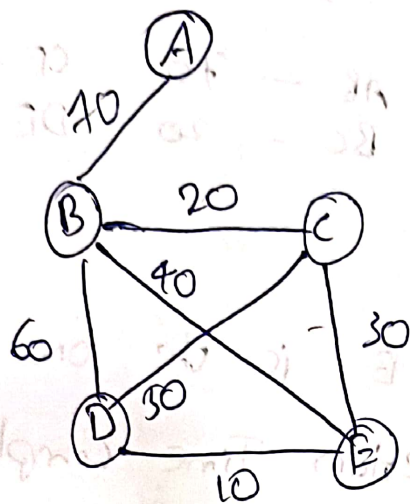
Applications of MST :-

① Image segmentation

② Computer Network

MST

① Kruskal Algorithm

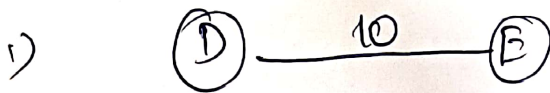


minimum cost of an edge

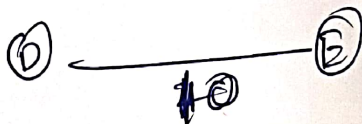
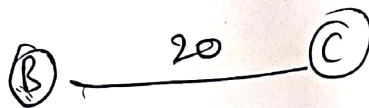
→ so Using ~~heap~~ edges, we will create one ^{min} heap

(10, 20, 30, 40, 50, 60, 70)

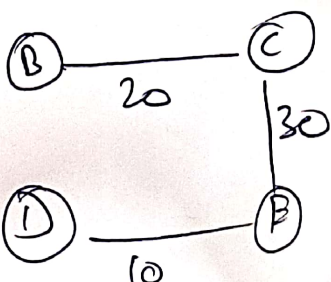
so, Using 10



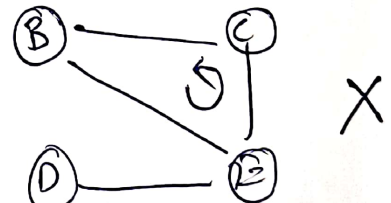
2) second minimum cost 20



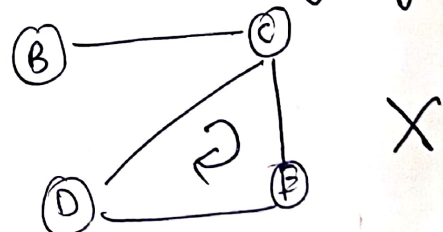
3) 30 is next cost



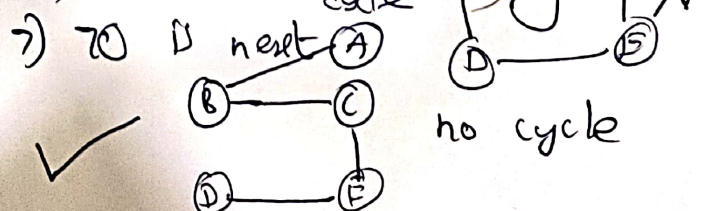
4) 40 is next, but if we connect 40 cycle is forming



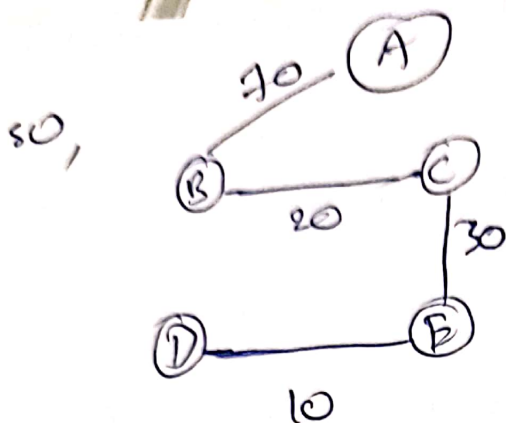
5) 50 also forming cycle



6) 60 is next form cycle



no cycle



$$\Rightarrow \text{edges} = (n-1) = (V-1)$$

$$10 + 20 + 30 + 70 = 130$$

$$AB - 70$$

$$BC - 20$$

$$CE \rightarrow 30$$

$$DE \rightarrow 10$$

heapq - document

Time Complexity

worst case itself

Log E is the one node deletion Time Complexity.

→ Totally, E edges are there. So,

$$\boxed{T.C = E \log E} \quad \text{or} \quad E \log V$$

Best Case Scenario :-

$$(V-1) \log E \Rightarrow \boxed{V \log V}$$